Submitted By: RISCV_ASSEMBLERS

Name: V. V. S. Aneesh
Roll Number: 2025122002

Name: M. Sai Poojith
Roll Number: 2025122010

Name: Anumay Rai
Roll Number: 2025122013

Submitted On

Date: 10 March 2026

The International Institute of Information Technology, Hyderabad

Contents

1 Introduction **3**

2 Pipeline Stages **4**

3 Pipeline Registers **4**
3.1 IF/ID Pipeline Register 5
3.2 ID/EX Pipeline Register 6
3.3 EX/MEM Pipeline Register 7
3.4 MEM/WB Pipeline Register 8

4 Hazard Handling Mechanisms **9**
4.1 Data Forwarding Unit 9
4.2 Hazard Detection Unit (Load-Use Stall) 10
4.3 Control Hazard Unit (Dynamic Branch Prediction) 11
4.4 Load-after-Store Forwarding 12

5 Test Cases and Waveforms **13**
5.1 Test Case 1: EX Hazard (MEM → EX Forwarding) 13
5.2 Test Case 2: MEM Hazard (WB → EX Forwarding) 14
5.3 Test Case 3: Load-Use Stall 15
5.4 Test Case 4: Dynamic Branch Prediction 15

6 Top-Level Pipelined Processor **16**

7 Testing and Results **20**
7.1 Test Program 20
7.2 Register File Output 21
7.3 Waveform Analysis 22

8 Contributions **22**

9 Conclusion **23**

Abstract

This project presents the design and implementation of a 5-stage pipelined RISC-V processor based on the RV64I instruction set, built upon the previously implemented sequential processor. The pipelined design improves instruction throughput by allowing multiple instructions to occupy different pipeline stages simultaneously: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB).

Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) are introduced between stages to hold data and control signals as instructions progress through the pipeline. To ensure correct execution in the presence of hazards, three mechanisms are implemented: a **Forwarding Unit** that resolves data hazards by bypassing results from the MEM and WB stages back to the EX stage, a **Hazard Detection Unit** that detects load-use hazards and stalls the pipeline when necessary, and a **Control Hazard Unit** that implements dynamic branch prediction using a 2-bit saturating counter Branch History Table (BHT).

As bonus features, the design also implements **Load-after-Store (LD-after-SD) forwarding** and **Write-Back to ID forwarding** in the register file to further reduce stalls. The processor is implemented in Verilog, simulated using iVerilog, and verified through test cases covering all hazard scenarios with waveform analysis.

1 Introduction

Sequential processors execute instructions one at a time, which limits the throughput of the processor. Pipelining improves performance by dividing instruction execution into multiple stages so that several instructions can be processed simultaneously.

In this project, the sequential RISC-V processor was extended to implement a 5-stage pipelined architecture consisting of:

- Instruction Fetch (IF)
- Instruction Decode (ID)
- Execute (EX)
- Memory Access (MEM)
- Write Back (WB)

Additional units were introduced to handle pipeline hazards, including a forwarding unit, hazard detection unit, and control hazard handling mechanisms.

The pipelined processor divides instruction execution into five stages. Pipeline registers are inserted between stages to hold intermediate data and control signals.

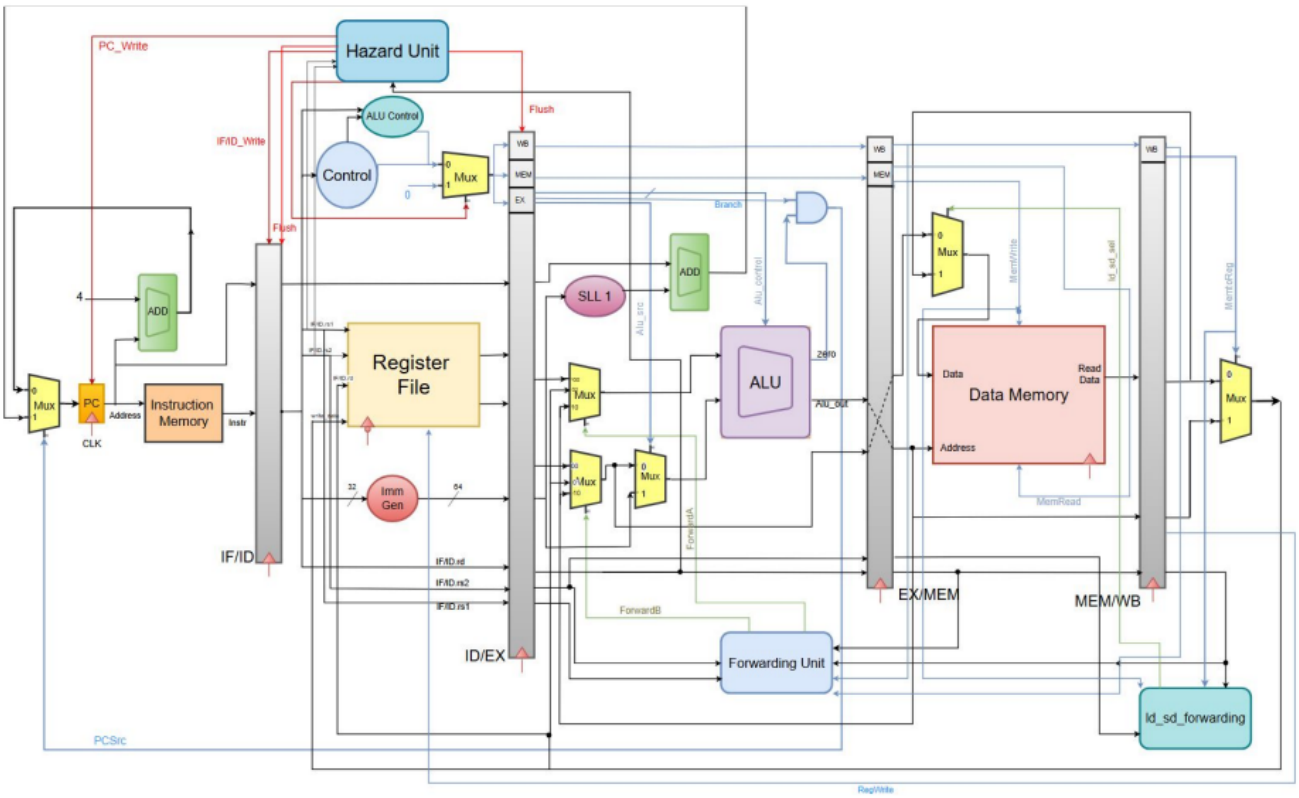

Figure 1: 5-stage pipelined RISC-V datapath

2 Pipeline Stages

A pipelined processor divides instruction execution into multiple stages so that several instructions can be processed simultaneously. This processor follows the classic five-stage pipeline consisting of Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB).

Instruction Fetch (IF)

In this stage, the instruction is fetched from instruction memory using the Program Counter (PC). The PC provides the address of the current instruction and is normally incremented by 4 to fetch the next instruction. The fetched instruction and PC value are stored in the IF/ID pipeline register.

Instruction Decode (ID)

The instruction is decoded in this stage and the Control Unit generates the required control signals. The register file reads the operands specified in the instruction, and the Immediate Generator produces the required immediate value. These values are passed to the next stage through the ID/EX pipeline register.

Execute (EX)

The Execute stage performs arithmetic and logical operations using the ALU. It also calculates branch target addresses and evaluates branch conditions. Forwarding logic is used to resolve data hazards by providing the latest operand values when required.

Memory Access (MEM)

In this stage, memory operations are performed. Load instructions read data from data memory, while store instructions write data to memory using the address computed by the ALU.

Write Back (WB)

The final stage writes the result of the instruction back to the register file. The value written may come from the ALU or from data memory depending on the instruction type.

3 Pipeline Registers

Pipeline registers separate adjacent pipeline stages and store intermediate data and control signals. They ensure that each stage receives stable inputs during every clock cycle while allowing multiple instructions to progress through the pipeline simultaneously.

3.1 IF/ID Pipeline Register

The IF/ID pipeline register is placed between the Instruction Fetch (IF) stage and the Instruction Decode (ID) stage. Its primary function is to store the instruction and the program counter value generated during the fetch stage and pass them to the decode stage in the next clock cycle.

This register helps isolate the pipeline stages so that multiple instructions can be processed simultaneously in different stages of execution.

The module stores two main values:

- The program counter value from the fetch stage
- The fetched instruction

The register updates its contents on the rising edge of the clock when the write signal is enabled. If the reset signal or flush signal is asserted, the register contents are cleared to zero. The flush signal is used during control hazard handling to discard incorrectly fetched instructions.

Implementation

Listing 1: Verilog Implementation of IF/ID Pipeline Register

```
module IF_ID(
    input reset,
    input clk,
    input flush,
    input IF_ID_write,
    input [63:0] IF_ID_pc_in,
    input [31:0] IF_ID_Ins_in,
    output [63:0] IF_ID_pc_out,
    output [31:0] IF_ID_Ins_out
);

    reg [63:0] IF_ID_pc_out_reg;
    reg [31:0] IF_ID_Ins_out_reg;

    always @(posedge clk) begin
        if (reset || flush) begin
            IF_ID_pc_out_reg <= 64'b0;
            IF_ID_Ins_out_reg <= 32'b0;
        end
        else if (IF_ID_write) begin
            IF_ID_pc_out_reg <= IF_ID_pc_in;
            IF_ID_Ins_out_reg <= IF_ID_Ins_in;
        end
    end

    assign IF_ID_pc_out = IF_ID_pc_out_reg;
    assign IF_ID_Ins_out = IF_ID_Ins_out_reg;

endmodule
```

3.2 ID/EX Pipeline Register

The ID/EX pipeline register is placed between the Instruction Decode (ID) stage and the Execute (EX) stage of the pipelined processor. Its purpose is to store intermediate data and control signals produced during the decode stage and forward them to the execute stage in the next clock cycle.

This register stores several important signals including register operands, immediate values, program counter values, control signals, and register addresses. These values are required by the execute stage to perform arithmetic or logical operations using the ALU.

The register updates its contents on the rising edge of the clock. If the reset signal is asserted, all stored values are cleared to zero. Additionally, a flush signal is provided to clear the pipeline register when hazards such as branch mispredictions occur. This mechanism helps discard invalid instructions and maintain correct pipeline execution.

Implementation

Listing 2: Verilog Implementation of ID/EX Pipeline Register

```
module ID_EX(
    input clk, reset, flush,
    input id_mem_to_reg, id_reg_write_en, id_mem_read,
    input id_mem_write, id_branch, id_alu_src,
    input [3:0] id_alu_ctrl,
    input [63:0] id_pc, id_data1, id_data2, id_imm,
    input [4:0] id_rs1, id_rs2, id_rd,
    output ex_mem_to_reg, ex_reg_write_en, ex_mem_read,
    output ex_mem_write, ex_branch, ex_alu_src,
    output [3:0] ex_alu_ctrl,
    output [63:0] ex_pc, ex_data1, ex_data2, ex_imm,
    output [4:0] ex_rs1, ex_rs2, ex_rd
);

    reg r_mt, r_rw, r_mr, r_mw, r_br, r_as;
    reg [3:0] r_ac;
    reg [63:0] r_pc, r_d1, r_d2, r_imm;
    reg [4:0] r_rs1, r_rs2, r_rd;

    always @(posedge clk or posedge reset) begin
        if (reset || flush) begin
            r_mt<=0; r_rw<=0; r_mr<=0; r_mw<=0; r_br<=0; r_as<=0;
            r_ac<=0; r_pc<=0; r_d1<=0; r_d2<=0; r_imm<=0;
            r_rs1<=0; r_rs2<=0; r_rd<=0;
        end
        else begin
            r_mt<=id_mem_to_reg; r_rw<=id_reg_write_en; r_mr<=id_mem_read;
            r_mw<=id_mem_write; r_br<=id_branch; r_as<=id_alu_src;
            r_ac<=id_alu_ctrl; r_pc<=id_pc; r_d1<=id_data1;
            r_d2<=id_data2; r_imm<=id_imm;
            r_rs1<=id_rs1; r_rs2<=id_rs2; r_rd<=id_rd;
        end
    end
```

```

    assign ex_mem_to_reg=r_mt; assign ex_reg_write_en=r_rw; assign ex_mem_read=
        r_mr;
    assign ex_mem_write=r_mw; assign ex_branch=r_br; assign ex_alu_src=r_as;
    assign ex_alu_ctrl=r_ac; assign ex_pc=r_pc; assign ex_data1=r_d1;
    assign ex_data2=r_d2; assign ex_imm=r_imm;
    assign ex_rs1=r_rs1; assign ex_rs2=r_rs2; assign ex_rd=r_rd;

endmodule
```

3.3 EX/MEM Pipeline Register

The EX/MEM pipeline register is positioned between the Execute (EX) stage and the Memory (MEM) stage of the pipelined processor. Its purpose is to store the results produced during the execute stage and transfer them to the memory stage in the next clock cycle.

This register stores the ALU result, data to be written to memory, destination register information, and control signals required for memory operations and register write-back. By holding these values, the EX/MEM register ensures correct data flow between the execute and memory stages while maintaining pipeline synchronization.

The register updates its contents on the rising edge of the clock. When the reset signal is asserted, all stored values and control signals are cleared to zero, bringing the pipeline stage to a known initial state.

Implementation

Listing 3: Verilog Implementation of EX/MEM Pipeline Register

```
module EX_MEM(
    input clk, reset,
    input ex_mem_to_reg, ex_reg_write_en, ex_mem_read, ex_mem_write,
    input [63:0] ex_alu_out, ex_store_data,
    input [4:0] ex_rs2, ex_rd,
    output mem_mem_to_reg, mem_reg_write_en, mem_mem_read, mem_mem_write,
    output [63:0] mem_alu_out, mem_store_data,
    output [4:0] mem_rs2, mem_rd
);

    reg r_mt, r_rw, r_mr, r_mw;
    reg [63:0] r_alu, r_sd;
    reg [4:0] r_rs2, r_rd;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            r_mt<=0; r_rw<=0; r_mr<=0; r_mw<=0;
            r_alu<=0; r_sd<=0; r_rs2<=0; r_rd<=0;
        end
        else begin
            r_mt<=ex_mem_to_reg; r_rw<=ex_reg_write_en;
            r_mr<=ex_mem_read; r_mw<=ex_mem_write;
```

```
            r_alu<=ex_alu_out; r_sd<=ex_store_data;
            r_rs2<=ex_rs2; r_rd<=ex_rd;
        end
    end

    assign mem_mem_to_reg=r_mt; assign mem_reg_write_en=r_rw;
    assign mem_mem_read=r_mr; assign mem_mem_write=r_mw;
    assign mem_alu_out=r_alu; assign mem_store_data=r_sd;
    assign mem_rs2=r_rs2; assign mem_rd=r_rd;
endmodule
```

3.4 MEM/WB Pipeline Register

The MEM/WB pipeline register is located between the Memory Access (MEM) stage and the Write Back (WB) stage of the pipelined processor. Its role is to store the outputs generated during the memory stage and pass them to the write-back stage in the next clock cycle.

This register stores the data read from memory, the ALU result, the destination register address, and the control signals required for the write-back stage. These values are then used to update the register file during the write-back stage.

The register updates its contents on the rising edge of the clock. If the reset signal is asserted, all stored data and control signals are cleared to zero to ensure that the pipeline starts from a known state.

Implementation

Listing 4: Verilog Implementation of MEM/WB Pipeline Register

```
module MEM_WB(
    input clk, reset,
    input wb_mem_to_reg_in, wb_reg_write_en_in,
    input [63:0] wb_mem_data_in, wb_alu_out_in,
    input [4:0] wb_rd_in,
    output wb_mem_to_reg, wb_reg_write_en,
    output [63:0] wb_mem_data, wb_alu_out,
    output [4:0] wb_rd
);

    reg r_mt, r_rw;
    reg [63:0] r_md, r_alu;
    reg [4:0] r_rd;

    always @(posedge clk or posedge reset) begin
        if (reset) begin
            r_mt<=0; r_rw<=0; r_md<=0; r_alu<=0; r_rd<=0;
        end
        else begin
            r_mt<=wb_mem_to_reg_in; r_rw<=wb_reg_write_en_in;
            r_md<=wb_mem_data_in; r_alu<=wb_alu_out_in;
            r_rd<=wb_rd_in;
```

```
        end
    end

    assign wb_mem_to_reg=r_mt; assign wb_reg_write_en=r_rw;
    assign wb_mem_data=r_md; assign wb_alu_out=r_alu;
    assign wb_rd=r_rd;

endmodule
```

4 Hazard Handling Mechanisms

Pipelining introduces hazards — situations where the next instruction cannot execute in the following clock cycle. This processor implements three categories of hazard handling: data hazard forwarding, load-use stall detection, and control hazard handling via dynamic branch prediction.

4.1 Data Forwarding Unit

A data hazard occurs when an instruction depends on the result of a previous instruction that has not yet written back to the register file. Without forwarding, the pipeline would have to stall until the result is available. The Forwarding Unit resolves this by bypassing the result directly from later pipeline stages back to the EX stage input.

Two forwarding cases are handled.

EX Hazard (MEM → EX Forwarding)

When the instruction currently in the MEM stage writes to a register that the instruction in the EX stage needs as a source operand, the ALU result from the EX/MEM pipeline register is forwarded directly to the ALU input of the EX stage. This eliminates pipeline stalls.

MEM Hazard (WB → EX Forwarding)

When the instruction in the WB stage writes to a register that the instruction in the EX stage requires, the value from the MEM/WB pipeline register is forwarded to the ALU input. This case is used when the EX hazard condition is not satisfied.

Listing 5: Forwarding Unit Implementation

```
module Forwarding_unit(
    input [4:0] ex_rs1, ex_rs2, mem_rd, wb_rd,
    input mem_regwrite, wb_regwrite,
    output [1:0] ForwardA, ForwardB
);
    reg [1:0] fa, fb;

    always @(*) begin
        fa = 2'b00;
        fb = 2'b00;

```

```
        if (mem_regwrite && (mem_rd != 5'b0) && (mem_rd == ex_rs1))
            fa = 2'b10;
        else if (wb_regwrite && (wb_rd != 5'b0) && (wb_rd == ex_rs1)
                && !(mem_regwrite && mem_rd == ex_rs1))
            fa = 2'b01;

        if (mem_regwrite && (mem_rd != 5'b0) && (mem_rd == ex_rs2))
            fb = 2'b10;
        else if (wb_regwrite && (wb_rd != 5'b0) && (wb_rd == ex_rs2)
                && !(mem_regwrite && mem_rd == ex_rs2))
            fb = 2'b01;
    end

    assign ForwardA = fa;
    assign ForwardB = fb;

endmodule
```

4.2 Hazard Detection Unit (Load-Use Stall)

Forwarding alone cannot resolve a **load-use hazard**. This occurs when a `ld` instruction is immediately followed by an instruction that uses the loaded value. Since the data becomes available only after the MEM stage, the next instruction cannot receive it in time for the EX stage.

The Hazard Detection Unit identifies this dependency and inserts a one-cycle stall (bubble) into the pipeline by:

- Freezing the Program Counter (PC)
- Freezing the IF/ID pipeline register
- Inserting a NOP bubble into the ID/EX pipeline register

Detection condition

$$\text{EX.MemRead} \wedge (\text{EX.rd} \neq 0) \wedge ((\text{EX.rd} = \text{ID.rs1}) \vee (\text{EX.rd} = \text{ID.rs2}))$$

Listing 6: Hazard Detection Unit Implementation

```
module hazard_detection_unit(
    input ex_mem_read, cur_mem_write, cur_mem_read,
    input [4:0] if_id_rs1, if_id_rs2, ex_rd,
    output pc_write, if_id_write, bubble_sel
);
    reg r_pcw, r_ifw, r_bub;

    always @(*) begin
        r_pcw = 1; r_ifw = 1; r_bub = 0;

        if (ex_mem_read && (ex_rd != 5'b0) &&
            ((ex_rd == if_id_rs1) ||
```

```
            (ex_rd == if_id_rs2 &&
             !(cur_mem_read || cur_mem_write)))) begin
           r_pcw = 0;
           r_ifw = 0;
           r_bub = 1;
        end
    end

    assign pc_write = r_pcw;
    assign if_id_write = r_ifw;
    assign bubble_sel = r_bub;

endmodule
```

4.3 Control Hazard Unit (Dynamic Branch Prediction)

Control hazards arise when branch instructions alter the normal sequential execution flow. Because the branch outcome is determined in the EX stage, the pipeline may have already fetched subsequent instructions that should not be executed.

To reduce the performance penalty, the processor implements **dynamic branch prediction** using a **2-bit saturating counter Branch History Table (BHT)**. The BHT contains 16 entries and is indexed using the program counter bits [5:2].

2-bit Saturating Counter States

State	Binary	Next State (Taken / Not Taken)
Strongly Not Taken	00	01 / 00
Weakly Not Taken	01	10 / 00
Weakly Taken	10	11 / 01
Strongly Taken	11	11 / 10

The counter increases when a branch is taken and decreases when it is not taken. This mechanism improves prediction stability for loop branches.

Flush Logic

The pipeline is flushed only when the prediction is incorrect:

- Predicted Not Taken but actually Taken
- Predicted Taken but actually Not Taken

Listing 7: Control Hazard Unit with 2-bit BHT

```
module control_hazard(
    input clk, reset,
    input branch_instr, branch_taken,
    input [63:0] branch_pc,
    output flush, predicted_taken
);
```

```
    reg [1:0] bht [0:15];
    integer idx;
    wire [3:0] bht_index = branch_pc[5:2];

    assign predicted_taken = bht[bht_index][1];

    always @(posedge clk or posedge reset) begin
        if (reset)
            for (idx = 0; idx < 16; idx = idx + 1)
                bht[idx] <= 2'b01;
        else if (branch_instr) begin
            case (bht[bht_index])
                2'b00: bht[bht_index] <= branch_taken ? 2'b01 : 2'b00;
                2'b01: bht[bht_index] <= branch_taken ? 2'b10 : 2'b00;
                2'b10: bht[bht_index] <= branch_taken ? 2'b11 : 2'b01;
                2'b11: bht[bht_index] <= branch_taken ? 2'b11 : 2'b10;
            endcase
        end
    end

    reg flush_reg;

    always @(*) begin
        flush_reg = 0;
        if (branch_instr) begin
            if (!predicted_taken && branch_taken)
                flush_reg = 1;
            else if (predicted_taken && !branch_taken)
                flush_reg = 1;
        end
    end

    assign flush = flush_reg;

endmodule
```

4.4 Load-after-Store Forwarding

A special data hazard occurs when a `ld` instruction is followed by a `sd` instruction that stores the loaded value. Without additional handling, the store instruction may use stale data from the register file.

The LD-after-SD forwarding unit detects this condition and forwards the loaded value from the MEM/WB stage directly to the memory write input.

Detection condition

$$\text{WB.MemToReg} \wedge \text{MEM.MemWrite} \wedge (\text{WB.rd} = \text{MEM.rs2}) \wedge (\text{WB.rd} \neq 0)$$

Listing 8: LD-after-SD Forwarding Unit

```
module ld_after_sd_forwarding(
    input [4:0] ld_rd, sd_rs2,
    input ld_mem_to_reg, sd_mem_write,
    output ld_sd_sel
);
    reg ld_sd_sel_reg;

    always @(*) begin
        ld_sd_sel_reg = 1'b0;

        if (ld_mem_to_reg && (ld_rd == sd_rs2) &&
            (ld_rd != 5'b0) && sd_mem_write)
            ld_sd_sel_reg = 1'b1;
    end

    assign ld_sd_sel = ld_sd_sel_reg;

endmodule
```

5 Test Cases and Waveforms

The following test cases verify each hazard handling mechanism implemented in the pipelined processor.

5.1 Test Case 1: EX Hazard (MEM → EX Forwarding)

Description

This test verifies that the forwarding unit correctly forwards the ALU result from the EX/MEM pipeline register to the ALU input of the following instruction, avoiding a stall.

Assembly Program

Listing 9: EX Hazard Test Program

```
addi x1, x0, 10      # x1 = 10
add  x2, x1, x1      # x2 = x1 + x1 = 20   (EX hazard: x1 from above)
add  x3, x2, x1      # x3 = x2 + x1 = 30   (EX hazard: x2 from above)
```

Expected Register Values

```
x1 = 0x000000000000000a  (10)
x2 = 0x0000000000000014  (20)
x3 = 0x000000000000001e  (30)
```

Waveform

Figure 2: Waveform showing EX hazard forwarding: ForwardA = 10 indicates result bypassed from EX/MEM register to ALU input

5.2 Test Case 2: MEM Hazard (WB → EX Forwarding)

Description

This test verifies forwarding from the MEM/WB pipeline register to the EX stage when the producing instruction is two cycles ahead.

Assembly Program

Listing 10: MEM Hazard Test Program

```
addi x1, x0, 5      # x1 = 5
addi x2, x0, 10     # x2 = 10   (independent)
add  x3, x1, x2     # x3 = 15   (MEM hazard: x1 from 2 cycles ago)
```

Expected Register Values

```
x1 = 0x0000000000000005  (5)
x2 = 0x000000000000000a  (10)
x3 = 0x000000000000000f  (15)
```

Waveform

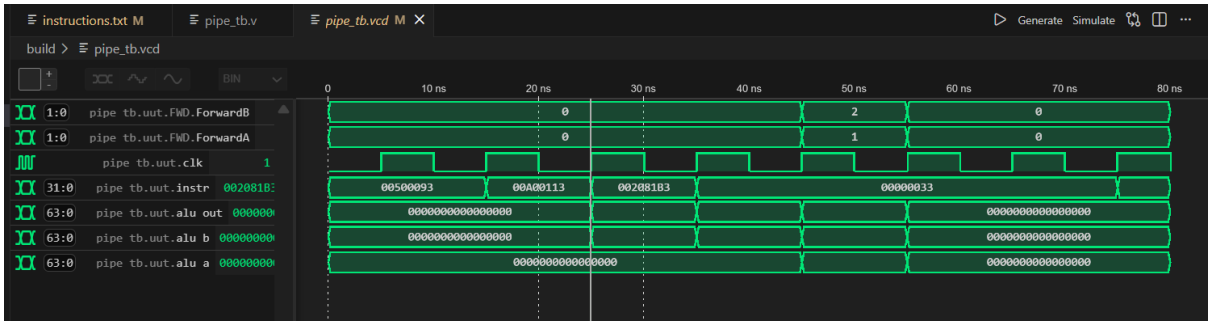

Figure 3: Waveform showing MEM hazard forwarding: ForwardA = 01 indicates result bypassed from MEM/WB register to ALU input

5.3 Test Case 3: Load-Use Stall

Description

This test verifies that the hazard detection unit correctly inserts a one-cycle stall when a `ld` instruction is immediately followed by an instruction that uses the loaded value.

Assembly Program

Listing 11: Load-Use Stall Test Program

```
addi x1, x0, 8      # x1 = 8  (memory address)
sd   x0, 0(x1)      # store 0 at address 8
addi x2, x0, 42     # x2 = 42
sd   x2, 0(x1)      # store 42 at address 8
ld   x3, 0(x1)      # x3 = 42 (load from address 8)
add  x4, x3, x3     # x4 = 84 (load-use hazard on x3!)
```

Expected Register Values

```
x1 = 0x0000000000000008  (8)
x2 = 0x000000000000002a  (42)
x3 = 0x000000000000002a  (42)
x4 = 0x0000000000000054  (84)
```

Waveform

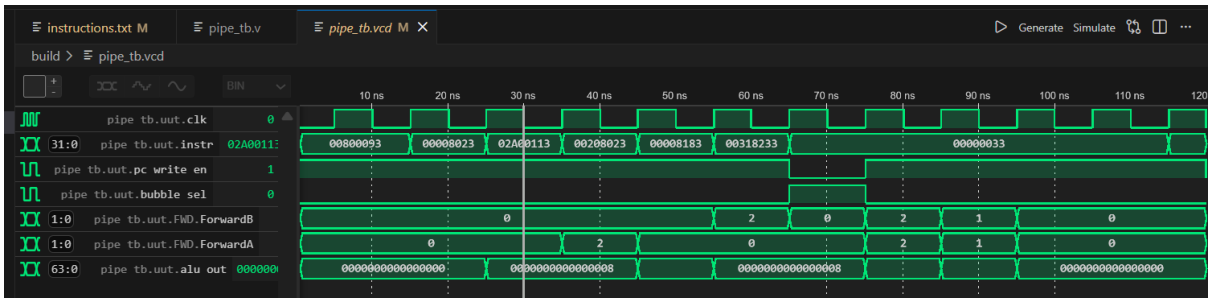

Figure 4: Waveform showing load-use stall: pc_write = 0 and if_id_write = 0 indicate pipeline freeze; bubble_sel = 1 indicates NOP insertion

5.4 Test Case 4: Dynamic Branch Prediction

Description

This test verifies the dynamic branch prediction mechanism using a simple loop. The branch is taken multiple times (loop iterations) and not-taken once (loop exit). The 2-bit BHT learns to predict taken after the first iteration, reducing flushes significantly.

Assembly Program

Listing 12: Branch Prediction Test Program (Loop)

```
addi x1, x0, 3      # x1 = 3 (loop counter)
addi x2, x0, 0      # x2 = 0 (accumulator)
# loop:
add  x2, x2, x1     # x2 += x1
addi x1, x1, -1     # x1--
beq  x1, x0, 8      # if x1==0, exit loop (branch forward)
beq  x0, x0, -12    # unconditional branch back to loop
# exit:
add  x3, x2, x0     # x3 = x2 = 6 (3+2+1)
```

Expected Register Values

```
x1 = 0x0000000000000000  (0, counter exhausted)
x2 = 0x0000000000000006  (6 = 3+2+1)
x3 = 0x0000000000000006  (copy of x2)
```

Waveform

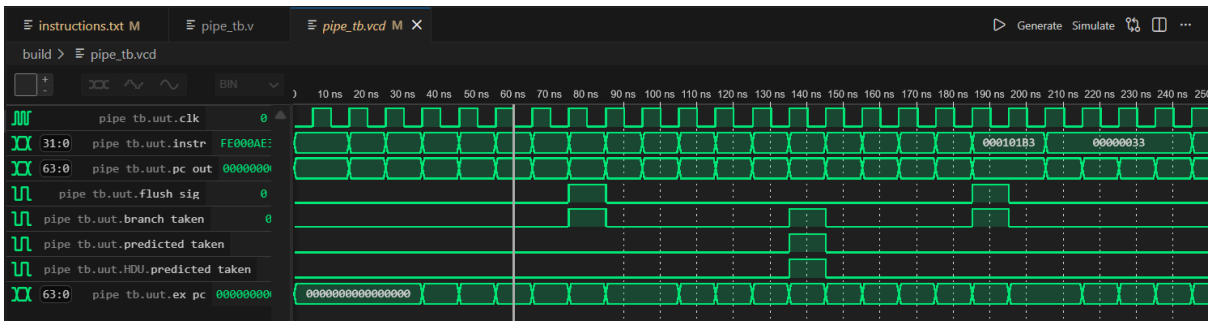

Figure 5: Waveform showing dynamic branch prediction: `flush_sig` asserted only on misprediction. `branch_taken` and `predicted_taken` confirm BHT learning behaviour.

6 Top-Level Pipelined Processor

The `pipe` module represents the complete implementation of the five-stage pipelined RISC-V processor. This module integrates all the individual components of the processor including the instruction fetch stage, decode stage, execute stage, memory stage, and write-back stage. It also connects all pipeline registers, forwarding units, and hazard detection mechanisms to ensure correct pipeline execution.

The processor pipeline consists of five stages:

- **Instruction Fetch (IF)** – Fetches instructions from instruction memory using the program counter.
- **Instruction Decode (ID)** – Decodes the instruction, generates control signals, reads register operands, and produces immediate values.

- **Execute (EX)** – Performs ALU operations and evaluates branch conditions.
- **Memory Access (MEM)** – Executes load and store operations using data memory.
- **Write Back (WB)** – Writes results from the ALU or memory back to the register file.

Pipeline registers (`IF/ID`, `ID/EX`, `EX/MEM`, and `MEM/WB`) are used to transfer intermediate data between pipeline stages. The design also includes a forwarding unit to resolve data hazards and a hazard detection unit to stall the pipeline when necessary. Control hazards caused by branch instructions are handled using a flush mechanism that clears incorrect instructions from the pipeline.

Implementation

Listing 13: Top-Level Implementation of the Pipelined RISC-V Processor

```
module pipe(input clk, reset);

    // IF
    wire [63:0] pc_out, pc_plus4, pc_branch, pc_next;
    wire [31:0] instr;
    wire pc_write_en, flush_sig, if_id_write_en;

    // IF/ID
    wire [63:0] if_id_pc;
    wire [31:0] if_id_ins;

    // ID: control
    wire branch, mem_read, mem_to_reg, mem_write, alu_src, reg_write;
    wire [1:0] alu_op;
    wire [3:0] alu_ctrl_raw;

    // ID: bubble outputs
    wire bubble_sel;
    wire b_branch, b_mem_read, b_mem_to_reg, b_mem_write, b_alu_src,
        b_reg_write;
    wire [3:0] b_alu_ctrl;

    // ID: register file
    wire [63:0] rf_rdata1, rf_rdata2;
    wire [63:0] wb_write_data;
    wire [4:0] wb_rd;
    wire wb_reg_write;

    // ID: imm gen
    wire [63:0] imm;

    // ID/EX
    wire ex_mem_to_reg, ex_reg_write, ex_mem_read, ex_mem_write;
```

```
    wire ex_branch, ex_alu_src;
    wire [3:0] ex_alu_ctrl;
    wire [63:0] ex_pc, ex_data1, ex_data2, ex_imm;
    wire [4:0] ex_rs1, ex_rs2, ex_rd;

    // EX: forwarding
    wire [1:0] fwd_a, fwd_b;
    wire [63:0] alu_a, alu_b_pre, alu_b;
    wire [63:0] alu_out;
    wire alu_zero;
    wire branch_taken;

    // EX/MEM
    wire mem_mem_to_reg, mem_reg_write, mem_mem_read, mem_mem_write;
    wire [63:0] mem_alu_out, mem_store_data;
    wire [4:0] mem_rs2, mem_rd;

    // MEM: ld-sd forwarding
    wire ld_sd_sel;
    wire [63:0] mem_write_data_final;
    wire [63:0] mem_read_data;

    // MEM/WB
    wire wb_mem_to_reg;
    wire [63:0] wb_mem_data, wb_alu_out;
    wire [4:0] wb_rd_sig;
    wire wb_reg_write_sig;

    // ==========================================================
    // IF STAGE
    // ==========================================================
    adder64 add4(.a(pc_out), .b(64'd4), .sum(pc_plus4));

    adder64 add_branch(.a(ex_pc), .b(ex_imm), .sum(pc_branch));

    and2 branch_and(.a(ex_branch), .b(alu_zero), .out(branch_taken));

    mux2_1 pc_mux(.a(pc_plus4), .b(pc_branch), .sel(branch_taken), .out(pc_next
        ));

    pc PC(.clk(clk), .reset(reset), .pc_write(pc_write_en),
          .pc_in(pc_next), .pc_out(pc_out));

    Instruction_Memory IMEM(.addr(pc_out), .instr(instr));

    IF_ID IFID(.reset(reset), .clk(clk), .flush(flush_sig),
               .IF_ID_write(if_id_write_en),
               .IF_ID_pc_in(pc_out), .IF_ID_Ins_in(instr),
               .IF_ID_pc_out(if_id_pc), .IF_ID_Ins_out(if_id_ins));

    // ==========================================================
```

```
    // ID STAGE
    // ==========================================================
    control CTRL(
        .opcode(if_id_ins[6:0]),
        .Branch(branch), .MemRead(mem_read), .MemToReg(mem_to_reg),
        .ALUOp(alu_op), .MemWrite(mem_write), .ALUSrc(alu_src),
        .reg_write_en(reg_write)
    );

    alu_control ALUCTRL(
        .ALUOp(alu_op),
        .Ins({if_id_ins[30], if_id_ins[14:12]}),
        .ALUControl(alu_ctrl_raw)
    );

    control_bubble BUBBLE(
        .branch_in(branch), .mem_read_in(mem_read), .mem_to_reg_in(mem_to_reg),
        .mem_write_in(mem_write), .alu_src_in(alu_src), .reg_write_in(reg_write),
        .alu_ctrl_in(alu_ctrl_raw), .sel(bubble_sel),
        .branch_out(b_branch), .mem_read_out(b_mem_read),
        .mem_to_reg_out(b_mem_to_reg), .mem_write_out(b_mem_write),
        .alu_src_out(b_alu_src), .reg_write_out(b_reg_write),
        .alu_ctrl_out(b_alu_ctrl)
    );

    register_file RF(
        .clk(clk), .reset(reset), .reg_write_en(wb_reg_write),
        .read_reg1(if_id_ins[19:15]), .read_reg2(if_id_ins[24:20]),
        .write_reg(wb_rd), .write_data(wb_write_data),
        .read_data1(rf_rdata1), .read_data2(rf_rdata2)
    );

    Immediate_Generation IMMGEN(.instr(if_id_ins), .imm(imm));

    ID_EX IDEX(
        .clk(clk), .reset(reset), .flush(flush_sig),
        .id_mem_to_reg(b_mem_to_reg), .id_reg_write_en(b_reg_write),
        .id_mem_read(b_mem_read), .id_mem_write(b_mem_write),
        .id_branch(b_branch), .id_alu_src(b_alu_src), .id_alu_ctrl(b_alu_ctrl),
        .id_pc(if_id_pc), .id_data1(rf_rdata1), .id_data2(rf_rdata2), .id_imm(imm),
        .id_rs1(if_id_ins[19:15]), .id_rs2(if_id_ins[24:20]), .id_rd(if_id_ins[11:7]),
        .ex_mem_to_reg(ex_mem_to_reg), .ex_reg_write_en(ex_reg_write),
        .ex_mem_read(ex_mem_read), .ex_mem_write(ex_mem_write),
        .ex_branch(ex_branch), .ex_alu_src(ex_alu_src), .ex_alu_ctrl(ex_alu_ctrl),
        .ex_pc(ex_pc), .ex_data1(ex_data1), .ex_data2(ex_data2), .ex_imm(ex_imm),
        .ex_rs1(ex_rs1), .ex_rs2(ex_rs2), .ex_rd(ex_rd)
```

```
    );

    // ==========================================================
    // EX STAGE
    // ==========================================================
    Forwarding_unit FWD(
        .ex_rs1(ex_rs1), .ex_rs2(ex_rs2),
        .mem_rd(mem_rd), .wb_rd(wb_rd_sig),
        .mem_regwrite(mem_reg_write), .wb_regwrite(wb_reg_write_sig),
        .ForwardA(fwd_a), .ForwardB(fwd_b)
    );

    assign alu_a = (fwd_a == 2'b10) ? mem_alu_out :
                   (fwd_a == 2'b01) ? wb_write_data : ex_data1;

    assign alu_b_pre = (fwd_b == 2'b10) ? mem_alu_out :
                   (fwd_b == 2'b01) ? wb_write_data : ex_data2;

    mux2_1 alu_src_mux(.a(alu_b_pre), .b(ex_imm), .sel(ex_alu_src), .out(alu_b)
        );

    alu_64_bit ALU(.a(alu_a), .b(alu_b), .opcode(ex_alu_ctrl),
                .result(alu_out), .zero_flag(alu_zero));

    EX_MEM EXMEM(
        .clk(clk), .reset(reset),
        .ex_mem_to_reg(ex_mem_to_reg), .ex_reg_write_en(ex_reg_write),
        .ex_mem_read(ex_mem_read), .ex_mem_write(ex_mem_write),
        .ex_alu_out(alu_out), .ex_store_data(alu_b_pre),
        .ex_rs2(ex_rs2), .ex_rd(ex_rd),
        .mem_mem_to_reg(mem_mem_to_reg), .mem_reg_write_en(mem_reg_write),
        .mem_mem_read(mem_mem_read), .mem_mem_write(mem_mem_write),
        .mem_alu_out(mem_alu_out), .mem_store_data(mem_store_data),
        .mem_rs2(mem_rs2), .mem_rd(mem_rd)
    );
endmodule
```

7 Testing and Results

To verify the correct functionality of the pipelined RISC-V processor, a comprehensive test program was executed that exercises arithmetic operations, logical operations, memory load/store, forwarding paths, and branch instructions simultaneously.

7.1 Test Program

Listing 14: Comprehensive Pipeline Test Program

```
addi x2, x0, 5          # x2 = 5
addi x3, x0, 10         # x3 = 10
```

```
add   x1, x2, x3        # x1 = x2 + x3 = 15    [EX Hazard: x2, x3]
sub   x2, x2, x3        # x2 = x2 - x3 = -5    [EX Hazard: x2, x3]
and   x4, x3, x3        # x4 = x3 & x3 = 10    [MEM Hazard: x3]
and   x5, x3, x4        # x5 = x3 & x4 = 10    [EX Hazard: x4]
or    x6, x2, x4        # x6 = x2 | x4 = -5    [MEM Hazard: x2, x4]
or    x7, x2, x3        # x7 = x2 | x3 = -5    [MEM Hazard: x2]
sd    x1, 0(x5)         # mem[10] = 15
ld    x10, 0(x5)        # x10 = mem[10] = 15    [Load-Use Stall]
sd    x6, 24(x5)        # mem[34] = -5
ld    x11, 24(x5)       # x11 = mem[34] = -5    [Load-Use Stall]
beq   x4, x5, 8         # x4==x5, branch taken  (skip next instruction)
beq   x0, x0, 0         # skipped (infinite loop avoided by branch above)
add   x13, x1, x10      # x13 = x1 + x10 = 30
add   x0, x0, x0        # NOP (pipeline flush)
add   x0, x0, x0        # NOP
add   x0, x0, x0        # NOP
add   x0, x0, x0        # NOP
```

7.2 Register File Output

The register file contents after execution are shown below. All values match the expected results from the execution trace.

```
Total cycles: 21

x0  = 0000000000000000   (hardwired zero)
x1  = 000000000000000f   (15  = 5 + 10)
x2  = fffffffffffffffb   (-5  = 5 - 10)
x3  = 000000000000000a   (10)
x4  = 000000000000000a   (10  = x3 & x3)
x5  = 000000000000000a   (10  = x3 & x4)
x6  = fffffffffffffffb   (-5  = x2 | x4)
x7  = fffffffffffffffb   (-5  = x2 | x3)
x10 = 000000000000000f   (15  = loaded from mem[10])
x11 = fffffffffffffffb   (-5  = loaded from mem[34])
x13 = 000000000000001e   (30  = x1 + x10 = 15 + 15)
All other registers = 0000000000000000
```

The forwarding unit successfully eliminated stalls for all EX and MEM data hazards. Only the two load-use hazards required pipeline stalls, as forwarding cannot resolve these without a one-cycle delay.

7.3 Waveform Analysis

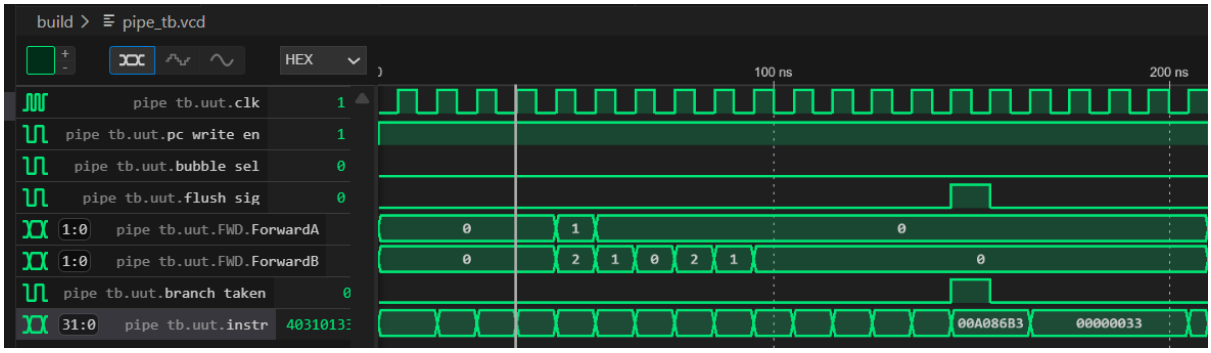

Figure 6: Full pipeline waveform showing key signals: `clk`, `pc_write_en`, `bubble_sel`, `flush_sig`, `ForwardA`, `ForwardB`, `branch_taken`, and `instr`.

8 Contributions

The pipelined RISC-V processor was developed collaboratively by all team members. The overall architecture, debugging, and testing were performed together through continuous discussions and joint problem-solving sessions. While each member primarily focused on certain modules of the design, the entire pipeline was integrated and verified collectively.

- **V. V. S. Aneesh**
  `IF/ID` Pipeline Register, Forwarding Unit, Top-Level Integration
- **M. Sai Poojith**
  `ID/EX` and `EX/MEM` Pipeline Registers, Hazard Detection Unit, Testbench and Verification
- **Anumay Rai**
  `MEM/WB` Pipeline Register, Dynamic Branch Prediction (2-bit BHT), Load-after-Store Forwarding

It should be noted that the above division of work does not fully reflect the nature of the development process. The processor was designed, implemented, and debugged as a collaborative effort throughout the project. All major design decisions were made through continuous discussion and collective problem solving among all team members. Each member actively contributed across all aspects of the implementation and possesses a thorough understanding of the complete codebase.

9 Conclusion

This project presents the successful design and implementation of a 5-stage pipelined RISC-V processor based on the RV64I instruction set. The pipelined architecture significantly improves instruction throughput over the previously implemented sequential processor by allowing multiple instructions to occupy different pipeline stages simultaneously: Instruction Fetch (IF), Instruction Decode (ID), Execute (EX), Memory Access (MEM), and Write Back (WB).

Pipeline registers (IF/ID, ID/EX, EX/MEM, MEM/WB) were introduced between stages to correctly propagate data and control signals across the pipeline. Data hazards were resolved through a combination of a **Forwarding Unit**, which bypasses results from the MEM and WB stages directly to the EX stage, and a **Hazard Detection Unit**, which inserts a one-cycle stall bubble when a load-use hazard is detected that cannot be resolved by forwarding alone.

Control hazards arising from branch instructions were handled using **dynamic branch prediction** with a 2-bit saturating counter Branch History Table (BHT). This approach reduces pipeline flushes by learning branch behaviour over repeated executions, outperforming static always-not-taken prediction for loop-heavy programs.

As bonus features, **Load-after-Store (LD-after-SD) forwarding** and **Write-Back to ID forwarding** were implemented to further reduce unnecessary stalls and improve pipeline efficiency.

Simulation results verified correct execution of all supported instructions including arithmetic, logical, memory load/store, and branch instructions. All hazard scenarios were validated through targeted test cases and waveform analysis, confirming the correctness of the hazard handling mechanisms.

References

[1] Sarah L. Harris and David Money Harris, *Digital Design and Computer Architecture: RISC-V Edition*, Morgan Kaufmann, 2021.

[2] David A. Patterson and John L. Hennessy, *Computer Organization and Design: RISC-V Edition*, Morgan Kaufmann, 2nd Edition, 2020.

[3] RISC-V Foundation, *The RISC-V Instruction Set Manual, Volume I: Unprivileged ISA*, Document Version 20191213, Available at: `https://riscv.org/technical/specifications/`

[4] LupLab, UC Davis, *RISC-V Instruction Encoder/Decoder*, Online Tool used for instruction encoding, Available at: `https://luplab.gitlab.io/rvcodecjs/`